

Supplementary Registers and Scenario Catalogue for Aura File Defender

Oleksandr Melnychenko

Khmelnytskyi National University, Khmelnytskyi, Ukraine

Abstract

This supplement contains the field-level evaluation policy, canonical format register, safe scenario catalogue, and reproducibility checklist referenced by the Aura File Defender methodology. It is versioned with the implementation and does not replace the definitions or claim boundaries in the main article.

Appendix A. Evaluation Policy Register

This appendix records the frozen policy dimensions. The versioned release document names a base profile and may override only schema-authorized top-level bounds and blocked extensions. Any other edit changes the implementation commit and creates a new run.

Appendix A.1. Registered profiles and evaluation selection

The implementation defines staging, strict, and paranoid profiles, with public, enterprise, and high-risk aliases. Staging can warn or quarantine and is only a rehearsal profile. Strict is the primary release base. It requires supplied type evidence, blocks unknown kinds and decoder failures, enforces output convergence, scans input and output, and requires semantic reconstruction for static images, GIF, audio, and video. Paranoid narrows budgets and selected output sets. A paranoid comparison is paired and not pooled with strict. Dry run, warning, bypass, and unmodified fallback paths never count as accepted.

Appendix A.2. Security-policy fields

Table [A.1](#) identifies whether each field is source-defined or a schema-authorized release-document value and states its evidence obligation.

Table A.1: Security-policy register and required evidence.

Field	Owner	Release rule	Required evidence
Profile and version	Release document	Known version and strict base	Parsed document, schema validation, SHA-256
Input bytes	Release document	Reject above bound before handler work	Cases below, at, and above L , with stage record
Output bytes	Release document	Terminate or reject above bound	Child-output and in-process boundary cases
Expansion ratio	Release document	Reject candidate ratio above bound	Constructible below, at, and above cases
Handler duration	Source	Earliest of policy and caller deadline	Expired, near-bound, timeout, and cancellation cases
Required MIME	Source	Present and recognized in strict profile	Missing, alias, unsupported, and conflicting cases
Type agreement	Source	Name, supplied type, bytes, output type, and parser converge	S3 crossing matrix and T3 report
Unknown kind	Source	Block	Unknown and unsupported S6 records
Decoder or probe failure	Source	Block in strict profile	Missing tool, invalid decode, timeout, and nonzero exit
Reconstruction level	Source by kind	Structural or semantic-or-block as registered	Per-record performed and required fields
Signature policy	Source and rule input	Input and output scan, blocking pre-scan in strict	Rule digest, location, severity, and stage result
Blocked extensions	Source plus release document	Normalized union, no silent deletion	Name vectors and serialized normalized policy
Output MIME allowlist	Source	Enforce per file kind	Allowed, denied, unknown, and cross-kind outputs
Image pixels	Source	Check before and after decode	Dimension and aggregate-pixel boundaries
Animation frames and pixels	Source	Bound frames, frame area, aggregate work, and duration	S5 generated boundary records

Field	Owner	Release rule	Required evidence
Audio duration, bitrate, channels	Source	Probe then enforce before semantic processing	Independent probe record and boundary cases
Video duration and bitrate	Source	Probe then enforce before semantic processing	Independent probe record and boundary cases
Container elements and nesting	Source validator	Exact grammar-specific budgets	S2 and S5 mutation records
External process protocols	Source	Local file and pipe protocols only	Command capture and prohibited-protocol regression
External process diagnostics	Source	Bounded and sanitized	Oversized output and control-character injection
Publication	Source	Atomic publication for Clean only	Fault campaign directory trace and hashes

Bounds use implementation units. Ratios derive from exact integer sizes. The serialized value, not a manuscript approximation, is authoritative.

Appendix A.3. Reconstruction register

The required level is checked against the handler report. Structural processing supports container and metadata claims but not neutralization inside retained compressed payloads. Semantic processing creates output from bounded decoded state and remains conditional on codec assumptions. Semantic-or-block forbids structural fallback.

Appendix A.4. Policy ordering for fixed-witness regression

The relation $p_2 \sqsubseteq p_1$ means that p_2 is at least as restrictive as p_1 under one implementation and parser-registry version. It requires no larger maximum bounds, no smaller minimum requirements, no weaker reconstruction, no broader output language, no smaller rejection relation, and no bypass in p_2 . The regression holds candidate bytes and parser reports fixed. A policy that produces another candidate is reported as a reconstruction change rather than a monotonicity result.

Paranoid $\sqsubseteq$ strict applies only after a generated policy diff. Staging is unordered because warnings and quarantine change terminal semantics.

Appendix A.5. Independent fidelity policy

Fidelity thresholds are held in a separate schema-valid document. That document maps each output profile to a metric profile, required tools, required structural comparisons, objective quality metrics, and pass thresholds. It also identifies documented transformations such as metadata removal, codec conversion, fixed application identifiers, or animation delay normalization. The document is frozen before corpus execution and its SHA-256 is included in the fidelity report.

Missing metrics, nonfinite values, decoder errors, and unevaluated accepted outputs fail fidelity. Thresholds are justified and frozen before results. No observed metric is asserted here.

Appendix A.6. Change control

Digests cover release, alias, fidelity, format, and tool configuration. Any post-run change invalidates the old interpretation and requires a new identifier and complete rerun of the controlled sequence in Figure 5.

Appendix B. Canonical Format Register

This appendix records the implemented output profiles against which the methods in Section 5 are interpreted. A registry entry does not state an empirical success rate. It states the accepted output language, reconstruction level, retained semantic boundary, and strongest claim that follows from implementation inspection. The registry version in the Rust validator is `aura-canonical-registry-v1`. MIME aliases in the table select one parser profile. They do not create additional grammars.

Table A.2: Evaluation reconstruction requirements and admissible claims.

Family	Strict requirement	Admissible accepted-output claim	Explicit exclusion
Static raster images	Semantic or block	New canonical image from bounded decoded pixels	Harmful visual meaning and steganographic content
GIF	Semantic or block	New bounded frame sequence with documented timing normalization	Arbitrary meaning encoded in frames
APNG	Semantic or block for high-risk claim	New semantic animation when available	Structural remix is insufficient for bitstream disarm
Animated WebP	Semantic or block for high-risk claim	New semantic animation when available	Structural RIFF reconstruction and complete frame decoding do not replace retained compressed frame payloads
MP3, WAV, FLAC, Ogg	Semantic or block in strict audio profile	New output from decoded audio representation	Structural copy cannot neutralize retained codec payload
MP4, MOV, M4A	Semantic or block in strict audio or video profile	Constrained semantic output after external decoding	Structural atom remix supports only container claims
WebM	Semantic or block in strict video profile	Constrained semantic output after external decoding	Structural EBML remix supports only container claims
Unsupported classes	Not applicable	No deliverable under media-only policy	No behavioral family classification

Table B.3: Implemented canonical output register and claim boundary.

Family and output type	Validator profile	Performed level	Canonical component closure	Retained representation	Strongest supported methodological claim
Static PNG, image/png	png-static-canonical-v1	Semantic	Signature, one IHDR, optional compatible PLTE and tRNS, contiguous IDAT, one IEND. Every CRC is checked.	Decoded pixel raster is re-encoded. Source PNG chunks and source compressed data are not copied.	Metadata exclusion, complete-boundary closure, static-profile enforcement, and semantic raster reconstruction under the selected decoder and encoder.
JPEG, image/jpeg	jpeg-canonical-v1	Semantic	SOI, pinned JFIF APP0, explicit marker allowlist, baseline SOF0 or progressive SOF2 structure, scan tables and entropy data, EOI. Other APP and COM markers are forbidden.	Decoded pixel raster is re-encoded as a new JPEG.	Metadata exclusion, marker and scan closure, suffix rejection, and semantic raster reconstruction. Lossy fidelity remains an empirical obligation.
Static WebP, image/webp	webp-static-canonical-v1	Semantic	RIFF/WEBP with exact size and padding, optional consistent VP8X, and exactly one VP8 or VP8L. Animation and metadata chunks are forbidden.	Decoded pixel raster is re-encoded as a new static WebP.	Metadata and animation exclusion, RIFF boundary integrity, and semantic raster reconstruction.
Animated WebP, image/webp	webp-animated-canonical-v1	Structural	RIFF/WEBP with exact size and padding, one animation-enabled VP8X, one ANIM, and bounded ANMF frames containing an optional ALPH and one VP8 or VP8L. Metadata and unknown chunks are forbidden.	Compressed frame payloads are retained. Every reconstructed frame is subsequently decoded under fixed bounds.	Metadata exclusion, canvas and frame-rectangle consistency, frame-count agreement, complete RIFF closure, and bounded decoder consumability. Codec-payload neutralization is not claimed.

Continued on next page

Family and output type	Validator profile	Performed level	Canonical component closure	Retained representation	Strongest supported methodological claim
GIF, image/gif	gif-canonical-v1	Semantic	Header, logical screen, generated color tables, generated frame controls and image blocks, optional generated loop extension, one trailer.	Decoded RGBA frames, geometry, disposal, transparency, and normalized delay are re-encoded.	Frame-level semantic reconstruction, extension exclusion, complete trailer consumption, and bounded frame workload.
APNG, image/apng	apng-canonical-v1	Structural	PNG signature, IHDR, optional required palette/transparency, one acTL , contiguous fcTL/fdAT sequence with IDAT , one IEND .	Compressed IDAT and fdAT payloads survive structural remuxing.	Chunk, CRC, frame-sequence, rectangle, metadata, and suffix claims only. Codec-payload neutralization is semantic-or-block and is not supplied by this path.
MP3, audio/mpeg	mp3-frame-sequence-v1	Structural	Nonempty sequence of complete MPEG audio frames with stable supported stream parameters. Tags, inter-frame material, and suffixes are forbidden.	MPEG frame bodies are retained.	ID3 and appended-material removal plus exact frame-boundary validation. Compressed audio disarm is not claimed.
WAV PCM, audio/wav and audio/x-wav	wav-pcm-canonical-v1	Structural	RIFF/WAVE with exact outer size, one 16-byte PCM fmt before one data , and only generated odd-byte padding.	PCM sample bytes are retained in a new container.	Ancillary-chunk exclusion, arithmetic consistency, data alignment, and complete RIFF reconstruction for supported integer PCM.
FLAC, audio/flac	flac-frame-crc-v1	Structural	fLaC , one 34-byte final STREAMINFO block, and one or more completely parsed frames with CRC-8 and CRC-16.	STREAMINFO content and compressed frames are retained.	Metadata exclusion, constrained frame grammar, frame-CRC integrity, numbering and STREAMINFO field consistency, and EOF closure. Decoded-sample MD5 and codec neutralization are not claimed.

Continued on next page

∞

Family and output type	Validator profile	Performed level	Canonical component closure	Retained representation	Strongest supported methodological claim
Ogg Opus or Vorbis, audio/ogg and audio/opus	ogg-single-stream-canonical-v1	Structural	One logical stream with valid pages, CRCs, serial and sequence continuity, coherent BOS/continuation/EOS, generated minimal comment, and media packets.	Identification and media packets are retained. The comment packet is replaced.	Single-stream closure, comment replacement, page and packet boundary integrity, and suffix rejection. Retained packet semantics are outside the claim.
ISO BMFF video, video/mp4	iso-bmff-constrained-v1	Structural, or semantic candidate followed by structural normalization	One profile-allowed ftyp , complete nonfragmented moov , one mdat , parent-specific allowlists, mandatory hierarchy, coherent sample tables, and exact sample-extent coverage of mdat .	Native mode retains encoded samples in mdat . A semantic tool path may replace them before normalization.	Container metadata removal, nonfragmented hierarchy closure, sample-table consistency, and exact media-data accounting. Native codec-payload neutralization is not claimed.
ISO BMFF audio, audio/mp4 , audio/x-m4a , and audio/m4a input aliases	iso-bmff-constrained-v1	Structural, or semantic candidate followed by structural normalization	Same constrained ISO BMFF hierarchy, selected for an audio track and supported audio sample entry.	Native mode retains encoded audio samples in mdat .	Same container and offset claims as video MP4. M4A naming and media-type aliases do not create a separate parser grammar.
WebM, video/webm	webm-constrained-v2	Structural	Complete EBML header and Segment, canonical Info, one or two constrained tracks, and SimpleBlock-only Clusters with parsed lacing and monotone timestamps.	VP8, VP9, AV1, and Opus frame payloads are retained.	WebM-only container closure, metadata and stale-index exclusion, track-description checks, frame extent parsing, timing bounds, and EOF closure. Codec-frame neutralization is not claimed.

Appendix B.1. Static raster register notes

BMP and TIFF are decodeable input classes in the semantic image path but are not canonical output profiles. Their accepted outputs are newly encoded PNG, JPEG, or static WebP according to policy. Static PNG explicitly forbids APNG chunks. Static WebP explicitly forbids animation chunks. These distinctions matter because a broad label such as “image” is not a release grammar. Raster validation combines a structural parser with a second bounded decode. The current validation decode permits at most 30 million pixels. This fixed validator bound applies in addition to the handler’s policy-bound probe and post-decode checks.

The metadata policy for semantic rasters is exclusion by reconstruction. The intermediate image object carries decoded pixels rather than source EXIF, XMP, ICC, comments, thumbnails, or application records. The output parser then verifies that the emitted grammar contains no component through which those classes could remain. This is a structural absence claim. It is not a claim that no pixel pattern can encode private or executable information under some external interpretation.

Appendix B.2. Animated image register notes

GIF, APNG, and animated WebP occupy different reconstruction levels despite all representing animation. GIF frames are decoded and re-encoded. The GIF method also normalizes delays below two centiseconds and enforces an aggregate decoded pixel budget. The fixed duration threshold of 6000 centiseconds applies to one animation cycle. It does not bound total viewing time for an indefinite loop. The registered semantic state therefore includes documented timing normalization, and exact second-pass byte equality is not expected.

APNG is a constrained PNG chunk language. The current path validates and remuxes complete chunks but retains compressed frame data. Required palette and transparency chunks are preserved only when compatible with IHDR. All other ancillary chunks are outside the output allowlist. A successful `apng-canonical-v1` report does not upgrade the performed level from **Structural** to **Semantic**. Under a semantic-or-block policy, the correct outcome remains blocking even when the candidate APNG is structurally canonical.

Animated WebP is a constrained RIFF chunk language. The handler removes ICCP, EXIF, and XMP, rewrites the outer length and feature bits, and retains bounded ANMF frame payloads. The independent validator reparses the entire chunk hierarchy and decodes every frame to check canvas, dimensions, alpha state, frame count, and pixel budgets. Decoder success is an additional postcondition, not evidence that source frame payloads were re-

generated. The reported reconstruction level therefore remains **Structural**, and a semantic-or-block policy does not release this candidate.

Appendix B.3. Audio register notes

The MP3 profile starts at the first retained MPEG frame and ends at the exact end of the last frame. ID3v2 and ID3v1 can be removed by the handler, but the canonical output grammar admits neither. Frame length is calculated from version, layer, bitrate, sample rate, and padding rather than by searching for the next sync pattern. This prevents an incidental sync-like byte sequence inside a frame from defining the structural boundary.

The WAV profile is limited to integer PCM with a 16-byte `fmt` body. Compressed WAV variants, extensible format records, and arbitrary ancillary chunks are not part of this entry. Odd payload length is represented by one generated zero byte. The pad is included in the outer RIFF size but excluded from the `data` chunk's declared sample length.

FLAC receives a structural claim despite deep frame parsing. Parsing a predictor and residual grammar to a checksum-valid frame boundary is not the same operation as decoding samples and creating new frames. The validator does not recompute STREAMINFO MD5 from uncompressed samples. This distinction prevents checksum verification from being misreported as semantic reconstruction.

The Ogg entry covers Opus and Vorbis only. It does not cover chained logical streams, multiplexed serial numbers, foreign codecs, or arbitrary comment repagination. The handler's current safe replacement case is a complete single-page comment packet that shares no page with another packet. More general layouts fail closed. The independent validator can reconstruct packet boundaries across pages, but that broader validation ability does not imply a broader rewriting implementation.

Appendix B.4. Container register notes

The ISO BMFF entry is intentionally nonfragmented and uses one permitted `mdat` extent. Fragment tables and general multi-extent relocation are not registered. A box name is not globally safe. Acceptance depends on the box's parent, order, multiplicity, version, and relationship with other tables. The validator therefore combines parent-specific closure with cross-table sample accounting. Derived sample extents from all tracks must cover the single `mdat` payload exactly without a gap or overlap. MOV and M4A are input naming or branding contexts. The registered outputs are selected as `video/mp4` or `audio/mp4` after track proof. Apple-originated `audio/m4a` and `audio/x-m4a` values are routing aliases for the latter. The structural

profile removes nonessential advisory and legacy atoms, including sample-group, sample-dependency, partial-sync, composition-shift, object-descriptor, extended-language, and track-reference records. Canonical visual sample entries may retain `fiel` and `chrm` only when their values and lengths satisfy the constrained validator.

The EBML implementation registers WebM, not unrestricted Matroska. A `DocType=matroska` input bypasses the native WebM rewriter and can enter the supervised semantic transcoder when policy requires and permits that path. It never inherits the WebM structural claim merely because both formats use EBML. The WebM profile permits one video track using VP8, VP9, or AV1 and at most one Opus track. Subtitle, metadata, and attachment tracks are excluded. Unknown-size encoding is accepted only for the root Segment, where EOF supplies the bound. SimpleBlock lacing is parsed to complete frame extents, while video lacing is excluded. SeekHead, Cues, Position, PrevSize, and related stale-location structures are removed rather than copied after layout changes.

Appendix B.5. Unsupported classes and release interpretation

No canonical output entry exists for PDF, Office documents, archives, executables, scripts, vector markup, packages, shortcuts, or unknown binary formats. Detection of a familiar prefix, extension, or MIME string does not promote one of these classes to support. Adding support would require a new output grammar, reconstruction-level decision, metadata policy, complete parser, post-reconstruction validator, resource model, and evidence plan.

The register also does not list AAC as an assigned format method, although the codebase contains an ADTS output validator. This appendix is scoped to the families specified for the present manuscript contribution and does not infer a complete reconstruction claim from validator registration alone. In all entries, a canonical report remains necessary but insufficient for publication. The performed level must satisfy policy, post-reconstruction checks must pass, and the release result must contain no blocking condition. Measured robustness, compatibility, fidelity, and performance remain deferred to the controlled server run.

Appendix C. Safe Scenario Catalogue

This catalogue defines non-operational carrier tests. Each identifier expands to format and boundary instances while retaining its S0–S7 stratum.

Executable-shaped material uses an ASCII sentinel, optional format magic, an inert length, and deterministic padding. It contains no code,

command, address, credential, persistence action, or exploit. Decoder-control-flow claims require S7 evidence instead.

Appendix C.1. Benign reconstruction and metadata scenarios

Table C.4: Benign and metadata scenario register.

ID	Stratum and mechanism	Safe construction	Expected action	Primary evidence
SC-01	S0 canonical minimum	Generate the smallest profile-valid object accepted by each concrete output grammar, with matching name and supplied type	Accept	T1/F1, T3/F3, T7/F7
SC-02	S0 canonical boundaries	Generate low, central, and high policy-valid dimensions, frames, channels, durations, and sizes	Accept	T6/F6 and usability
SC-03	S1 privacy metadata	Insert synthetic author, location, device, comment, thumbnail, and application fields at valid locations	Accept if removable, otherwise block	T2/F2, T4/F4
SC-04	S1 optional structure	Add legal but excluded ancillary chunks, boxes, tags, comments, padding, or vendor extensions	Accept if canonical reconstruction exists, otherwise block	T2/F2, T5/F5
SC-05	S1 alternate legal ordering	Reorder only elements whose source grammar permits the alternate order, preserving reference-parser acceptance	Accept if normalizable, otherwise block	T2/F2
SC-06	S1 semantic normalization	Generate GIF delays or output-codec inputs that trigger a documented normalization	Accept	T5/F5 and fidelity record

SC-01 and SC-02 do not establish safety of all valid media. SC-03 uses parsed field absence. SC-04 and SC-05 separate broad source validity from canonical output. SC-06 records deliberate change under the fidelity policy.

Appendix C.2. Malformed, disagreement, and multi-grammar scenarios

Table C.5: Malformed, classifier, boundary, and polyglot scenario register.

ID	Stratum and mechanism	Safe construction	Expected action	Primary evidence
SC-07	S2 truncated element	Cut a base object at every parsed header, payload, checksum, and terminator boundary	Block	T1/F1, T8/F8
SC-08	S2 inconsistent size	Change outer and inner lengths independently to below, exact, above, overflow-adjacent, and overlapping values	Block	T1/F1, T2/F2

ID	Stratum and mechanism	Safe construction	Expected action	Primary evidence
SC-09	S2 duplicate mandatory element	Duplicate IHDR, IEND, STREAMINFO, format, data, track identifier, or analogous required singleton	Block or reconstruct one unambiguous canonical object	T2/F2
SC-10	S2 checksum or sequence fault	Corrupt CRC, frame checksum, page sequence, animation sequence, or timestamp ordering while retaining the outer signature	Block	T2/F2, T8/F8
SC-11	S3 misleading final suffix	Pair valid media bytes and supplied media type with a forbidden or conflicting final extension	Block	T3/F3
SC-12	S3 hidden dangerous suffix	Use a basename such as an inert double-suffix vector whose nonfinal suffix belongs to an active class	Block	Name policy and T3/F3
SC-13	S3 supplied-type disagreement	Cross each byte profile with an incompatible, unknown, malformed, missing, and accepted alias media type	Block except the pre-specified equivalent alias	T3/F3
SC-14	S3 malformed display name	Insert path separators, dot components, control characters, bidirectional controls, device names, trailing dots or spaces, and overlength names	Block	Name policy and T8/F8
SC-15	S4 appended or trailing object	Append one inert foreign object after the canonical terminator or declared container boundary	Block or remove only through explicitly complete reconstruction	T1/F1
SC-16	S4 prefix camouflage	Prefix valid media with an inert executable-shaped or foreign-format sentinel	Block	T1/F1, T3/F3
SC-17	S4 true polyglot	Use a generated whole octet string independently and completely accepted from offset zero to file length by two incompatible parsers	Block	T3/F3 and differential validation
SC-18	S4 nested active object	Place an inert active-object sentinel in an attachment, metadata, or container location excluded by the canonical allowlist	Remove through canonical reconstruction or block	T2/F2 and T4/F4 for metadata locations

SC-07 through SC-10 instantiate Table 8. SC-09 accepts only after complete parsing and canonical singleton output. SC-11 through SC-14 use inert evidence. Their Trojan, downloader, dropper, or stager relevance concerns disguise only. SC-15 is not annotated as a true polyglot without complete whole-string acceptance under a second incompatible parser. SC-17 requires two parser reports whose terminal offsets both equal the file length. SC-18

requires parsed location, not a byte-pattern match.

Appendix C.3. Resource and unsupported-class scenarios

Table C.6: Resource, active-class, and regression scenario register.

ID	Stratum and mechanism	Safe construction	Expected action	Primary evidence
SC-19	S5 scalar policy limit	Generate $L - 1$, L , and $L + 1$ inputs for bytes, dimensions, frames, duration, bitrate, channels, elements, and nesting	Accept at admissible points, block above	T6/F6 and boundary trace
SC-20	S5 output expansion	Use a compact valid input whose semantic output lies below, at, or above the frozen byte and ratio limits	Accept below or at, block above	T6/F6 and output accounting
SC-21	S5 parser work budget	Generate long chains of zero-semantic-value chunks, atoms, pages, lacing segments, or nested elements within and beyond count budgets	Accept within if canonical, block beyond	T6/F6 and bounded parser trace
SC-22	S5 deadline and cancellation	Apply pre-cancelled, expired, near-deadline, and mid-process cancellation contexts to deterministic generated media	No deliverable after cancellation or expiry	T6/F6, T8/F8
SC-23	S6 active or unsupported class	Use non-executing fixtures with executable, script, archive, document, PDF, vector, shortcut, package, and unknown signatures and matching names	Block	T8/F8 and media-only policy
SC-24	S6 disguised active class	Cross an unsupported inert fixture with an allowed media suffix and media type	Block	T3/F3
SC-25	S7 parser regression	Use a legally distributable minimized case with affected-version evidence and a fixed expected result	Block or safely reconstruct as pre-specified	Regression and differential record
SC-26	S7 decoder regression	Run a minimized decoder-triggering case through the pinned isolated semantic path with process limits	No crash propagation and no invalid deliverable	T6/F6, T8/F8, and tool trace

At-limit SC-19 cases pass only if grammar and all other bounds hold. SC-20 keeps blocked candidates out of clean-output expansion. SC-22 holds the base digest constant. SC-23 and SC-24 establish unsupported-carrier blocking, not virus, worm, Trojan, ransomware, or spyware classification, and not downloader, dropper, or stager attribution. SC-25 and SC-26 remain specific to pinned cases and versions.

Appendix C.4. Fault and publication scenarios

Table C.7: Fault-injection and publication scenario register.

ID	Injection	Observation	Expected action	Primary evidence
SC-27	Child launch failure	Replace the configured external executable with a controlled nonexistent path	Typed failure, no deliverable	T8/F8
SC-28	Child timeout	Use a controlled adapter that exceeds the frozen timeout and responds to process-group termination	Typed timeout, no deliverable	T6/F6, T8/F8
SC-29	Diagnostic flood	Emit standard output and error beyond the capture bound with controls and bidirectional markers	Bounded sanitized diagnostic, no deliverable	T6/F6 and fault report
SC-30	Output growth	Make the controlled child grow its monitored temporary output beyond the byte bound	Child terminated, no deliverable	T6/F6, T8/F8
SC-31	Canonical validation failure	Substitute a deterministic noncanonical candidate after reconstruction in the instrumented runner	Block, candidate never published	T7/F7, T8/F8
SC-32	Handler panic	Trigger a checked regression hook that panics with a bounded attacker-shaped message	Typed pipeline error, bounded summary, no deliverable	T8/F8
SC-33	Non-clean publication request	Invoke the publication boundary with blocked or suspicious status in the fault adapter	Destination absent or unchanged	T8/F8 and publication invariant
SC-34	Pre-existing destination	Place a hashed benign sentinel at the final path before each failing operation	Original digest preserved	T8/F8 and atomicity trace

Fault scenarios use controlled hooks. SC-31 and SC-32 stay outside effectiveness denominators. Success requires no new deliverable and preservation of existing content. Allocator, creation, and persistence faults remain uncounted until explicit hooks exist.

Appendix C.5. Scenario expansion and audit trail

Each record names its scenario and records generator commit, base digest, profile, element path, changed values, seed, and expected reason. The artifact is hashed and not regenerated during measurement.

Coverage counts scenario-format pairs. An uninstantiated row is a gap, not a passing zero-event test.

Appendix D. Reproducibility and Release Provenance

Reproduction is defined by a source revision, locked dependency graph, policy document, classifier registry, corpus manifest, external-tool set, execu-

tion environment, and machine-readable result directory. The Rust toolchain is pinned to version 1.96.1 with a minimal profile plus formatting and linting components. `Cargo.lock` fixes Rust dependency resolution. `Cargo.toml` fixes unwinding panics in development and release because panic containment is part of the runtime contract. Rebuilding with another toolchain, unlocked dependencies, altered panic behavior, or an unrecorded policy is a different experimental condition.

The development repository is publicly available.

<https://github.com/oleksandrmelnichenko/ecliptix-file-defender>

The clean implementation snapshot inspected for this manuscript revision is `0bffc39314bc8ee840aa6a85e69f108e667fa5fd`. This identifier records the local source state but is not substituted for the commit field of a future controlled run. Submission still requires an immutable archive, its digest, and a persistent archive identifier. Until those exist, the repository is an inspectable development artifact rather than an exactly reproducible published artifact.

The ordinary quality workflow checks formatting, builds and tests every workspace target and feature, rejects lint warnings, packages every crate, and runs dependency advisory and license/source policy gates. Research scripts are syntax checked and JSON inputs are parsed. A macOS job rebuilds the Apple XCFramework, compares the generated distribution, header, and Swift package metadata against the committed artifact, then builds and tests the Swift package. A benchmark smoke job checks that the harness executes. These jobs are release preflight. They are not substitutes for the controlled corpus, fuzzing, fault-injection, fidelity, and performance run.

The controlled runner is restricted to a Linux validation server. It requires an explicit CPU affinity, records available frequency-governor state, and executes ten independent end-to-end benchmark sessions. Corpus order is deterministically permuted by session. Raw JSON Lines are retained before deterministic reduction. The run also records tool versions, host properties, commands, logs, accepted outputs, and a SHA-256 inventory. A development laptop may be used for compilation and narrow static inspection, but values obtained there do not populate manuscript result macros.

The corpus evaluator validates each manifest path, size, and digest before calling the public pipeline. It records the concrete input evidence, handler level, verdict, alerts, canonical report, output digest, and release-gate failures for each sample. Every clean output is processed a second time under its canonical name and media type. Exact byte equality is required only for deterministic structural and lossless paths. Lossy semantic paths report same-profile canonical closure separately. They are not described as byte-idempotent without a pre-specified normalized comparator and drift bound.

Table D.8: Provenance items required for a scientifically interpretable run.

Item	Required record	Integrity anchor
Source and build	Commit, clean-tree state, toolchain, target, features, panic profile	Git identifier and lockfile hash
Policy and registries	Release policy, fidelity policy, classifier aliases, canonical registry version	SHA-256 for each frozen input
Corpus	Sample identifier, stratum, origin, size, expected treatment, and path	Manifest plus per-sample SHA-256
External tools	FFmpeg, FFprobe, independent parsers, metric tools, and adapters	Version output and executable hash
Host state	Operating system, CPU, memory, storage, affinity, governor, and command line	Run manifest and environment record
Observations	Per-sample results, accepted outputs, raw benchmark records, coverage, fuzz logs, and faults	Schema validation and checksum inventory
Analysis	Deterministic scripts, thresholds, session grouping, and generated tables	Script hashes and analysis-summary schema

Three external adapters are mandatory for scientific release. The differential adapter evaluates every accepted output with at least two independently implemented parser or decoder families. The fidelity adapter applies a separately frozen threshold policy so usability does not silently redefine the security verdict. The fault-injection adapter covers pre-dispatch cancellation, expired deadlines, child-launch failure, child timeout, bounded diagnostics, output-size termination, canonical-validation failure, and blocked publication. Each adapter exits nonzero on incomplete coverage, tool failure, disagreement, or threshold failure and writes its report atomically. A tooling rehearsal that disables external evidence cannot pass the scientific release gate.

Apple release artifacts follow a separate reproducible packaging path. The build script compiles locked static libraries for iOS device, both simulator architectures, Mac Catalyst, and macOS. It combines architectures with `lipo`, creates one XCFramework, validates its property list, verifies that the strict FFI symbol exists in every archive, and compares every bun-

dled header with the source header. The script sets fixed archive epochs, remaps workspace and package-cache paths embedded by the Rust compiler, and sorts and canonically serializes the XCFramework property list. The CI comparison then detects byte drift in the checked-in binary target. These controls address the known path, timestamp, ordering, and serialization sources of nondeterminism. They do not imply byte-identical output across arbitrary unpinned Rust, Xcode, SDK, or archive-tool versions, which remain part of the recorded build environment.

Workspace publication is ordered because the facade depends on five internal crates. The leaf crates are published before `file_defender`, with registry-index propagation allowed between steps. Live publication requires a registry token, while dry run uses workspace packaging for the first release where internal crates may not yet exist in the registry. Dependency advisories and license/source policy are checked immediately before publication. These controls establish release provenance and packaging consistency. They do not establish absence of vulnerabilities in dependencies.

Numerical result macros in the master document remain **pending** until a run manifest exists. Every frozen run is registered and retained whether it passes or fails. A rerun receives a new identifier and links its predecessor and reason. Only release claims require every scientific release gate to pass. Tests, coverage-guided fuzzing, fault injection, and benchmarks are intentionally deferred to that controlled server run. Consequently, this appendix specifies commands, artifacts, and decision rules but reports no latency, throughput, memory, neutralization, false-blocking, or fidelity values. This boundary prevents local rehearsal output from being mistaken for evidence and keeps all claims tied to a reviewable provenance record.